PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

PERTEMUAN KE-2

CLASS DAN OBJECT

TUJUAN PRAKTIKUM

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. menerapkan konsep dasar kelas dan objek ke dalam program
2. menerapkan konstruktor dan membuat overloading-nya.
3. menerapkan konsep enkapsulasi data pada program

TOOLS

Tools yang diperlukan untuk melakukan praktikum ini adalah Java Development Kit (JDK) untuk mengkompilasi dan menjaalankan program Java serta code editor atau IDE untuk menulis program Java.

LANDASAN TEORI

Class merupakan template dari objek yang memiliki atribut dan perilaku serupa. Objek merupakan instansiasi dari sebuah class. Objek diciptakan menggunakan konstruktor dari class-nya.

Enkapsulasi bertujuan untuk membungkus data (atribut) dan method yang memanipulasi data tersebut dalam satu blok kelas. Enkapsulasi bertujuan untuk mendukung *information hiding*, sehingga data hanya bisa diakses oleh method-method di dalam kelas tersebut. Enkapsulasi umumnya diimplementasikan dengan memberikan ***access modifier private*** pada data/atribut untuk memastikan data tersebut hanya bisa diakses dalam satu blok kelas, serta memberikan ***access modifier public*** pada konstruktor dan method lainnya yang diijinkan untuk diakses dari luar kelas. Karena atribut di-set private, maka kelas dilengkapi dengan method selektor dan mutator untuk menyediakan akses ataupun manipulasi ke nilai atribut kelas.

LANGKAH PRAKTIKUM

Bagian 1 - Membuat Class dan Objek Titik

1. Tuliskan nama file, deskripsi, pembuat, dan tanggal pada bagian awal file Anda sebagai komentar.
2. Buatlah sebuah class Titik yang memiliki atribut berupa absis dan ordinat serta memiliki beberapa method seperti contoh berikut:

```java
/* Nama File     : Titik.java
 * Deskripsi     : berisi atribut dan method dalam class Titik
 * Pembuat       :
 * Tanggal       :
 */

public class Titik {
    /***************ATRIBUT***************/
    double absis;
    double ordinat;

    /***************METHOD***************/
    //konstruktor untuk membuat titik (0,0)
    Titik(){
        absis = 0;
        ordinat = 0;
    }

    //mengembalikan nilai absis
    double getAbsis(){
        return absis;
    }

    //mengembalikan nilai ordinat
    double getOrdinat(){
        return ordinat;
    }

    //mengeset absis titik dengan nilai baru x
    void setAbsis(double x){
        absis = x;
    }

    //mengeset ordinat titik dengan nilai baru y
    void setOrdinat(double y){
        ordinat = y;
    }

    //menggeser nilai absis dan ordinat titik masing-masing sejauh x dan y
    void geser(double x, double y){
        absis = absis + x;
        ordinat = ordinat + x;
    }

    //mencetak koordinat titik
    void printTitik(){
        System.out.println("Titik (" + absis + "," + ordinat + ")");
    }
} //end class Titik
```

3. Buatlah sebuah main class bernama MTitik yang menggunakan class Titik untuk mebuat objek Titik dan memanggil method yang telah dibuat dalam class Titik, seperti contoh berikut:

```java
/* Nama File     : Titik.java
 * Deskripsi     : berisi atribut dan method dalam class Titik
 * Pembuat       :
 * Tanggal       :
 */
public class MTitik {
    public static void main(String[] args) {
        Titik T1 = new Titik(); //Membuat objek titik T1 (0,0)
        T1.setAbsis(3);  //mengubah absis T1 dengan nilai 3
        T1.setOrdinat(4); //mengubah ordinat T1 dengan nilai 4
        T1.printTitik(); //mencetak koordinat T1 ke layar
        T1.geser(3,4); //menggeser T1 sejauh (3,4)
        T1.printTitik(); //menampilkan koordinat T1 setelah digeser
    }
}
```

Jalankan class MTitik dan amati hasilnya.

Bagian 2 – Object Reference

4. Tambahkan kode berikut dalam method main di dalam class MTitik sebagai berikut:

```java
        Titik T2 = T1;
        T2.printTitik();
        T1.setAbsis(10);
        T1.setOrdinat(10);
        T2.printTitik();
```

Jalankan program tersebut dan amati hasilnya. Apakah koordinat titik T2 ikut berubah saat nilai absis dan ordinat titik T1 diubah? Mengapa?

Bagian 3 – Overloading Constructor

5. Pada contoh nomor 2 konstruktor class Titik selalu menghasilkan titik (0,0). Dengan prinsip *overloading*, tambahkanlah sebuah konstruktor pada class Titik yang menerima masukan nilai absis dan ordinat untuk menghasilkan titik baru sesuai dengan nilai absis dan ordinat tersebut.

Bagian 4 – Static Attribute & Method

6. Tambahkanlah sebuah atribut static counterTitik untuk menyimpan jumlah objek titik yang telah diinstantiasi dari class Titik. Lalu, modifikasi konstruktor Titik dan tambahkan method static getCounterTitik() untuk mengembalikan nilai counterTitik, seperti berikut:

```java
13  public class Titik {
14      /***************ATRIBUT***************/
15      double absis;
16      double ordinat;
17      static int counterTitik = 0;
18
19      /***************METHOD***************/
20      //konstruktor untuk membuat titik (0,0)
21      Titik(){
22          absis = 0;
23          ordinat = 0;
24          counterTitik++;
25      }
26
27      //mengembalikan nilai counterTitik
28      static int getCounterTitik(){
29          return counterTitik;
30      }
```

7. Panggillah method getCounterTitik di dalam method main dari class MTitik, kemudian amati hasilnya.

```java
14          Titik T1 = new Titik(); //Membuat objek titik T1 (0,0)
15          Titik T2 = new Titik(3,5); //Membuat objek titik T2 (3,5)
16
17          System.out.println("Jumlah Objek Titik = " + Titik.getCounterTitik());
            System.out.println("Jumlah Objek Titik = " + T2.getCounterTitik());
```

Bagaimana perbedaan antara atribut class yang dideklarasi menggunakan keyword static dengan atribut class biasa (tanpa keyword static)?

Bagian 5 – Keyword This

8. Keyword this dalam Java memiliki beberapa kegunaan antara lain: sebagai reference ke dirinya sendiri untuk menunjuk ke atribut, method, ataupun konstruktor. Perhatikan contoh berikut, lalu praktikkan dan amati hasilnya.

```java
13  public class Titik {
14      /***************ATRIBUT***************/
15      double absis;
16      double ordinat;
17      static int counterTitik = 0;
18
19      /***************METHOD***************/
20      //konstruktor untuk membuat dengan nilai absis dan ordinat tertentu
21      Titik(double absis, double ordinat){
22          this.absis = absis;
23          this.ordinat = ordinat;
24          counterTitik++;
25      }
26
27      //konstruktor untuk membuat titik (0,0)
28      Titik(){
29          this(0,0);
30      }
```

9. Bagaimana jika this digunakan untuk mengakses atribut counterTitik seperti berikut?

```java
void printCounterTitik(){
    System.out.println(this.counterTitik);
}
```

Bagian 6 – Finalisasi Method Lainnya

10. Lengkapilah method-method lain yang berkaitan dengan class Titik, sehingga class Titik memiliki semua method seperti class diagram berikut:

Titik
- absis: real - ordinat: real
+ Titik() + Titik(absis: real, ordinat:real) + getAbsis(): real + getOrdinat(): real + setAbsis(x:real) + setOrdinat(y:real) + geser(x:real, y:real) + getKuadran(): integer + getJarakPusat(): real + getJarak(T: Titik): real + refleksiX() + refleksiY() + getRefleksiX(): Titik + getRefleksiY(): Titik

Catatan: prosedur refleksiX() langsung mengubah absis dan ordinat titik setelah direfleksikan terhadap sumbu X, sedangkan fungsi getRefleksiX() menghasilkan sebuah titik baru yang merupakan hasil refleksi sumbu X dari sebuah objek titik.

TUGAS/LATIHAN

Buatlah class Garis dalam program Java, lalu buatkan sebuah main class MGaris yang menggunakan class Garis tersebut.

1. Atribut Garis terdiri atas titik awal dan titik akhir yang masing-masing bertipe Titik. Tambahkan pula atribut static counterGaris untuk menyimpan banyaknya objek garis yang pernah dibuat.
2. Method Garis terdiri atas:
   a. Konstruktor tanpa parameter yang menginisialisasi titik awal dengan (0,0) dan titik akhir dengan (1,1).
   b. Konstruktor dengan parameter masukan titik awal dan titik akhir.
   c. Selektor (getter) untuk setiap atribut beserta mutatornya (setter). Tambahkan pula selektor untuk mendapatkan atribut static counterGaris.
   d. Method untuk mendapatkan panjang sebuah garis.
   e. Method untuk mendapatkan gradien dari sebuah garis.
   f. Method untuk mendapatkan titik tengah dari sebuah garis.
   g. Method untuk mengecek apakah garis tersebut sejajar dengan sebuah garis lainnya, mengembalikan true jika sejajar dan false jika sebaliknya.
   h. Method untuk mengecek apakah garis tersebut tegak lurus dengan sebuah garis lainnya, mengembalikan true jika sejajar dan false jika sebaliknya.
   i. Method untuk menampilkan ke layar titik awal dan titik akhir garis.

j. Method untuk menampilkan persamaan garis dalam bentuk string y = mx + c.
Contoh: garis yang dibuat dari titik (-2,0) dan (0,4) mempunyai persamaan garis y = 2x + 4.

Bagian 7 - Enkapsulasi

1. Gunakan kembali file class Titik dan Garis beserta main class-nya, yaitu MTitik dan MGaris yang telah dibuat pada praktikum pertama. Simpan pada folder atau package yang baru untuk praktikum kedua.
2. Tuliskan nama file, deskripsi, pembuat, dan tanggal pada bagian awal file Anda sebagai komentar.
3. Tambahkan access modifier private pada semua atribut di dalam class Titik, lalu tambahkan access modifier public pada semua method yang ada di dalam class Titik.

```
13  public class Titik {
14      /**************ATRIBUT**************/
15      private double absis;
16      private double ordinat;
17      private static int counterTitik = 0;
18
19      /**************METHOD***************/
20      //konstruktor untuk membuat titik (0,0)
21
22
23      //konstruktor untuk membuat dengan nilai absis dan ordinat tertentu
        public Titik(double absis, double ordinat){
25          this.absis = absis;
26          this.ordinat = ordinat;
27          counterTitik++;
28      }
```

4. Pastikan class Titik memiliki selektor untuk setiap atributnya (absis, ordinat, dan counterTitik) serta memiliki mutator untuk atribut absis dan ordinat. Mutator untuk atribut counterTitik tidak diperlukan karena nilai atribut tersebut hanya diubah melalui konstruktor Titik pada saat ada objek yang diciptakan.
5. Pada class MTitik, buatlah sebuah objek dari class Titik serta panggillah semua atribut dan method dari objek tersebut, lalu jalankan dan amati hasilnya. Apakah perbedaan pemanggilan atribut dan method dari objek Titik pada praktikum pertama (sebelum ditambahkan access modifier) dengan praktikum ini (setelah ditambahkan access modifier)?
6. Lakukan eksperimen terhadap access modifier, misalnya ubah access modifier konstruktor dan beberapa method di class Titik menjadi private, lalu jalankan kembali class MTitik, sedemikian hingga praktikan dapat memahami perbedaan penggunaan antara access modifier public dan private.
7. Lakukan hal yang sama terhadap class Garis. Tambahkan access modifier private pada semua atributnya dan access modifier public pada semua methodnya.
8. Pada class MGaris, buatlah sebuah objek dari class Garis serta panggillah semua atribut dan method dari objek tersebut, lalu jalankan dan amati hasilnya. Apakah pemanggilan method dapat memberikan hasil sesuai yang diharapkan?

Jika tidak, periksalah kembali body method pada class Garis. Class Garis memiliki atribut yang bertipe Titik. Karena semua atribut Titik telah di-set menjadi private, maka atribut Titik tidak dapat dipanggil secara langsung di dalam class Garis. Pengaksesan nilai atribut kelas harus dilakukan melalui selektornya, sedangan manipulasi nilai atribut kelas harus dilakukan melalui mutatornya.

PELAPORAN

Pada akhir sesi praktikum, laporkan hasil praktikum pada link http://tiny.cc/pbo26lap

Lengkapi semua file program yang harus dikerjakan dalam modul ini dan kumpulkan hasilnya kepada Asisten Praktikum via github. Oleh karena itu pastikan semua file yg dikerjakan selama praktikum sudah di-push di github.

**********Selamat Mengerjakan dan Berlatih **********